

Tirth Shah

Hartford, CT | (860) 924-9190 | tirthshahh28@gmail.com | linkedin.com/in/tirth-shah14 | github.com/TirthShahh28 | tirthshahh28.github.io

SKILLS

Programming: Python, C#, TypeScript, JavaScript, Java, SQL, C++, Bash, Shell Scripting

Backend: ASP.NET MVC, .NET Framework, Web API, FastAPI, Node.js, Flask, Django, SQLAlchemy, REST APIs

Frontend: React, Next.js, AngularJS, Bootstrap, HTML5, CSS3, Redux, Tailwind CSS

Databases: SQL Server, T-SQL, PostgreSQL, MySQL, MongoDB, Redis, Entity Framework, Prisma, Airflow

Cloud DevOps: Docker, Kubernetes, AWS (EC2, S3), Git, GitHub Actions, Visual Studio, IIS, CI/CD, Linux

Testing: pytest, Jest, Playwright, Cypress, unit testing

Engineering: OOP, Design Patterns, MVC, MVVM, DAL, SDLC, TDD, System Design, JWT, OAuth2, RBAC

Practices: Agile Scrum, requirements gathering, technical docs, code reviews, production support

AI & ML: scikit-learn, PyTorch, Hugging Face, LangChain, RAG

EXPERIENCE

Software Engineer – Capital Spirits

Jun 2025 - Mar 2026 | Hartford, CT

- Developing catalog and checkout on the customer-facing e-commerce website using **C#** services, **ASP.NET MVC**, and **SQL Server**, with REST integration to search indexing for 30,000+ SKUs across 3 stores; tuned **T-SQL** and data-access paths so catalog lookups dropped from about 4s to about 30ms.
- Implementing checkout on the public site with **Stripe** callbacks, server-side price checks, **JWT** sessions, and transactional **SQL Server** updates (including CT tax rules), using idempotent webhook handling and duplicate-order guards so payment disputes stopped recurring at the register.
- Maintaining the internal **ASP.NET MVC** admin portal for operations: 5 core areas (inventory, pricing, orders, sync status, user access), CRUD screens, **Entity Framework** reads/writes, and role-based SQL reports used daily by staff at all 3 locations.
- Replacing 6 recurring spreadsheet exports with SQL views and small admin modules on the portal, so pricing, tax, and stock questions pull from one **SQL Server** source instead of ad-hoc files that drifted from production data.
- Building nightly **T-SQL** jobs and **.NET** batch utilities to sync website and admin inventory, with validation rules and email alerts on failed feeds, so store and admin screens stay aligned without manual stock checks each morning.
- Documenting release checklists and walkthrough notes with ops before storefront and admin deploys using **Git** and **Visual Studio**, covering stock holds, tax, and payment retries so sign-off did not depend on engineering memory.
- Triaging defects on the website and admin portal via **IIS** logs, application traces, and **SQL Server** query review, then patching through **Git** with smoke tests before high-traffic weekends.

Software Engineer – Digital for Humanity

Jan 2022 - Apr 2024 | Remote

- Building **Python**, **SQL**, and **Airflow** ETL pipelines over 6M+ rows with **Pandas** validation and scheduled batches, feeding reporting tables used by leadership without manual file prep.
- Supporting internal **C#** tools on **SQL Server** so analysts could filter and export the same datasets the ETL jobs produced, which cut duplicate production queries and one-off database pulls.
- Maintaining **ASP.NET MVC** and **Web API** features with **Entity Framework**, **AngularJS**, and **Bootstrap** on admin pages: CRUD flows, export filters, and form validation fixes from user tickets on the remote team.
- Adding schema checks and **GitHub Actions** CI/CD so bad batches failed in staging, with stderr logs analysts could read before weekly dashboards and **Flask** downstream jobs picked up stale data.
- Translating analyst requests into tickets with acceptance criteria before **ASP.NET MVC** view or **T-SQL** report changes, which made remote standups shorter and UAT steps repeatable.
- Documenting shared-table assumptions when **Python** pipelines and **ASP.NET MVC** screens touched the same **SQL Server** objects, keeping three downstream exports consistent after schema changes.

Software Engineer – University of Connecticut

Sep 2024 - May 2026 | Storrs, CT

- Building **Python**, **Pandas**, and **SQL** automation for accommodation and exam scheduling (1,000+ students), with **pytest** coverage, replacing spreadsheet workflows and reducing manual triage volume for staff.
- Developing ETL jobs with schema validation in **PostgreSQL** and campus **SQL** stores so bad rows never reached audit dashboards, which removed most end-of-week reconciliation passes for three teams.
- Benchmarking **scikit-learn** routing models with precision-recall review, improving routing accuracy about 25% and reducing misrouted cases that had been driving repeat manual fixes.
- Shipping **Python** exception dashboards and SLA reports used in compliance cycles, surfacing exam-window conflicts before they reached proctors.

PROJECTS

RoomLock: Concurrent Booking API | FastAPI, PostgreSQL, SQLAlchemy, Stripe, Docker | GitHub

- Designing a REST API with PostgreSQL, Alembic migrations, JWT role-based access, and OpenAPI documentation for reservations, authentication, and payment paths used in interview-style system design practice.
- Implementing row-level locking and explicit booking states to prevent double-booking under concurrent requests, plus webhook verification and a background job to expire abandoned holds.

DocuLens: Document Ingestion API | FastAPI, PostgreSQL, Docker, structured logging

- Building a document ingestion service with chunked uploads, metadata tags, and SQL-backed batch status so operators can see which files passed validation, failed, or are still queued.
- Adding Docker health checks and structured logging with retry-friendly batch steps so one bad file does not block the rest of the queue and failures are traceable from API logs to the stored error row.

EDUCATION

University of Connecticut (GPA: 3.6/4.0)

Master of Engineering, Computer Science

Storrs, CT, US

Aug 2024 - May 2026